PROJECT REPORT

End-to-End Automated Data Cleaning & Reporting System

Submitted by: **Aditya Mishra**

BCA Student, Lovely Professional University (Expected 2027)

Date: 6 November 2026

GitHub Repository: https://github.com/Adi-dot0/automated-data-cleaning-reporting

1. Abstract

This project is a Python-based application that automates the complete data preparation workflow for a data analyst. A user uploads a raw CSV or Excel file in a Streamlit web interface, and the system automatically profiles the data, cleans it, validates its quality, shows interactive charts, and provides a cleaned, business-ready dataset for download along with a data quality report. The system was tested on a sample customer dataset containing duplicates, missing values, invalid emails, a negative amount and mixed date formats. After cleaning, the dataset improved from 15 rows with 5-6 missing values and 2 duplicate rows to 13 clean rows with zero missing values and a data quality score of 96.9 out of 100.

2. Business Problem

Data analysts spend a large part of their working time on repetitive data preparation tasks instead of actual analysis. For almost every new file, the same steps are repeated:

- Removing duplicate records
- Handling missing values
- Correcting inconsistent data formats and extra spaces
- Standardizing column names
- Identifying incorrect values such as invalid emails or negative amounts
- Preparing summary reports and basic visualizations

Doing these steps manually in Excel is slow and error-prone, and the process has to be repeated for every new file. This project automates that entire workflow into a single application.

3. Objectives

- Accept raw CSV and Excel files through a simple web interface
- Automatically profile any dataset (size, duplicates, missing values, data types)
- Clean data automatically with user-controlled options for missing values
- Validate data quality and give a single quality score out of 100
- Generate interactive charts and a downloadable quality report
- Provide the cleaned dataset as CSV and Excel downloads
- Record every processing run in a database for tracking

4. Technology Stack

Tool	Use in this project
Python	Core programming language
Pandas	Data manipulation and cleaning
NumPy	Numerical operations (used by Pandas)

Streamlit	Web user interface
Plotly	Interactive visualizations
OpenPyXL	Reading and writing Excel files
SQLite	Stores the processing history table
GitHub	Version control and project hosting

Note: The original project specification listed PostgreSQL as an optional database. SQLite is used in this implementation so that the application runs on any computer with no database setup. The processing_history table follows the same design given in the specification.

5. System Architecture

Raw Excel / CSV File → Streamlit File Upload → File Validation → Data Profiling → Data Cleaning → Data Quality Validation → Charts and Report Generation → Download Cleaned Dataset

The code is organized in modules, so each stage of the workflow is a separate, reusable Python file. This makes the project easy to understand, test and extend.

6. Modules of the System

Module 1 - File Upload (modules/file_upload.py)

Accepts CSV and Excel files, supports multiple file uploads, validates the file extension, checks that the file contains readable data, and displays the dataset dimensions.

Module 2 - Data Profiling (modules/data_profiling.py)

Automatically calculates total records, total columns, duplicate records, missing values per column, unique values, column data types, numerical statistics and categorical value distributions.

Module 3 - Automated Data Cleaning (modules/data_cleaning.py)

Standardizes column names to lowercase with underscores, removes extra spaces from text, treats blank strings as missing values, converts mostly-numeric text columns to numbers, removes duplicate rows, fills numeric missing values with mean or median and text missing values with mode (the user chooses the option in the interface), and converts date columns to a single YYYY-MM-DD format.

Module 4 - Data Quality Engine (modules/data_validation.py)

Applies validation rules: email format check, missing value counts, duplicate detection, negative value checks, date validity checks, and flags columns with more than 50 percent missing values. It generates a data quality score that starts at 100 and is reduced for missing values, duplicates and invalid emails.

Module 5 - Automated Reporting (modules/report_generation.py)

Generates the data quality report text containing the profile summary, the cleaning log, and the validation results after cleaning. It also prepares the cleaned Excel file for download.

Module 6 - Database (database/db_connection.py)

Every run is saved in a processing_history table with the file name, total records, duplicate records, missing values, processing status and processing time.

7. How to Run the Project

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

The application opens in the browser (normally at http://localhost:8501). Upload any CSV or Excel file, or use the sample file provided in the sample_data folder, select the cleaning options, and click Clean Data to see the profile, charts, quality score and download options.

8. Testing and Results

Unit tests

Six unit tests were written in the tests folder (test_cleaning.py and test_validation.py) covering column name cleaning, duplicate removal, missing value filling, quality score calculation, invalid email detection and duplicate detection. All 6 tests pass.

Test on sample data

The sample file sample_messy_data.csv was created with known problems: duplicate rows, missing age, city and purchase values, two invalid emails, one negative purchase amount, extra spaces in names and column headings, and dates written in three different formats. The verified results of running the system on this file are:

Measure	Before cleaning	After cleaning
Total rows	15	13
Duplicate rows	2	0
Missing values	5 (plus 1 blank)	0
Invalid emails	2 (flagged)	2 (flagged in report)
Negative amounts	1 (flagged)	1 (flagged in report)
Data quality score	-	96.9 / 100

Invalid emails and negative values are reported to the user instead of being silently changed, because the correct value cannot be known from the data alone. All other problems were corrected automatically, and a cleaned dataset and quality report were downloaded successfully in CSV, Excel and text formats.

9. Limitations

- Date conversion is applied to columns with 'date' in the column name, and email validation to columns with 'email' in the name

- Mixed date formats such as 05/02/2024 can be ambiguous (day-first or month-first)

- Very large files will be slow in Streamlit; the system is designed for normal analyst-size files

- Incorrect values like invalid emails are flagged, not corrected, because the true value is unknown

10. Future Scope

This project is the first of a three-project Data Analyst Automation Portfolio. The planned next projects are: (1) an Automated Business Intelligence and Dashboard Refresh Pipeline using Python, SQL and Power BI, and (2) an Automated Web Scraping and CRM Synchronization system using Python and the HubSpot API. Within this project, future improvements include support for more file formats, custom validation rules set by the user, and deployment on Streamlit Community Cloud.

11. Conclusion

The Automated Data Cleaning and Reporting System successfully converts a raw, messy dataset into a clean, validated and documented dataset through a simple web interface. It implements all the main stages of the data preparation workflow - upload, profiling, cleaning, validation, visualization, reporting and download - in modular, tested Python code. Testing on a sample dataset with known problems showed that duplicates and missing values were fully removed, data quality problems were correctly flagged, and the final quality score reached 96.9 out of 100. The complete project is available on GitHub at https://github.com/Adi-dot0/automated-data-cleaning-reporting

Appendix - Project Files

app.py (main Streamlit application); modules/file_upload.py, data_profiling.py, data_cleaning.py, data_validation.py, report_generation.py; database/db_connection.py; sample_data/sample_messy_data.csv; reports/cleaned_sample.csv and quality_report_sample.txt; tests/test_cleaning.py and test_validation.py; requirements.txt; README.md.